

vuln_lkm_cli.pdf

How the Vuln-LKM userspace CLI works

Repository: github.com/mayday5000/Vuln-LKM

Binary: vuln_lkm_cli (gcc -Wall -O2)

Device: /dev/vuln_lkm

Shared ABI: vuln_lkm.h

Educational material for the Vuln-LKM loadable kernel module. Not an exploit guide. Integer wrap and a bounded-struct string copy are described as they appear in the source.

1. Role of the CLI

vuln_lkm_cli is ordinary userspace. It is not a kernel module, not a udev helper, not a test framework. It opens /dev/vuln_lkm, fills the structs defined in vuln_lkm.h, and calls ioctl(2). That is the entire contract. The kernel module never sees argv, never sees the interactive menu, never sees the base64 dump. Those exist so a human can watch the ABI on the way out.

The program is built by the same Makefile that builds the .ko: gcc -Wall -O2 -o vuln_lkm_cli vuln_lkm_cli.c. It includes vuln_lkm.h with __KERNEL__ undefined, so the userspace half of the header pulls sys/ioctl.h and typedefs __u32 as uint32_t. If you rebuild the header's ioctl numbers without rebuilding the CLI, the command words will not match the module and you will get ENOTTY.

There are two front ends in one binary. argc == 1 enters an interactive loop. argc >= 2 is a one-shot get|set|add|sub|str. -h and --help print help and exit without opening the device. Every successful or attempted ioctl prints the command word in hex, the integer or string payload, and a base64 encoding of the struct bytes.

2. The shared header as the ABI

The CLI does not invent ioctl numbers. It uses the macros from vuln_lkm.h. Those macros expand at compile time to 32-bit command words. On Linux, _IOR and _IOW pack direction, type (magic 0x1B), ordinal, and sizeof the third argument.

```

1  #ifndef VULN_LKM_H
2  #define VULN_LKM_H
3
4  #ifdef __KERNEL__
5  #include <linux/ioctl.h>
6  #include <linux/types.h>
7  #else
8  #include <sys/ioctl.h>
9  #include <stdint.h>
10 typedef uint32_t __u32;
11 #endif
12
13 #define VULN_LKM_NAME "vuln_lkm"
14 #define VULN_LKM_IOC_MAGIC 0x1B
15 #define VULN_LKM_STR_MAX 256
16 #define VULN_LKM_KBUF 32
17
18 struct vuln_lkm_int {
19     __u32 value;
20 };
21
22 struct vuln_lkm_str {
23     __u32 len;
24     char data[VULN_LKM_STR_MAX];
25 };
26
27 #define VULN_LKM_GET _IOR(VULN_LKM_IOC_MAGIC, 1, struct vuln_lkm_int)
28 #define VULN_LKM_SET _IOW(VULN_LKM_IOC_MAGIC, 2, struct vuln_lkm_int)
29 #define VULN_LKM_ADD _IOW(VULN_LKM_IOC_MAGIC, 3, struct vuln_lkm_int)
30 #define VULN_LKM_SUB _IOW(VULN_LKM_IOC_MAGIC, 4, struct vuln_lkm_int)
31 #define VULN_LKM_STR _IOW(VULN_LKM_IOC_MAGIC, 5, struct vuln_lkm_str)
32
33 #endif

```

Expected packing on a typical 64-bit Linux userspace (dir in bits 30–31, size in 16–29, type in 8–15, nr in 0–7):

- VULN_LKM_GET = _IOR(0x1B, 1, struct vuln_lkm_int) → 0x80041b01 (read, 4 bytes)

- VULN_LKM_SET = _IOW(0x1B, 2, struct vuln_lkm_int) → 0x40041b02 (write, 4 bytes)
- VULN_LKM_ADD = _IOW(0x1B, 3, struct vuln_lkm_int) → 0x40041b03
- VULN_LKM_SUB = _IOW(0x1B, 4, struct vuln_lkm_int) → 0x40041b04
- VULN_LKM_STR = _IOW(0x1B, 5, struct vuln_lkm_str) → 0x41041b05 (write, 260 bytes)

The CLI prints ioctl cmd: 0x%x of whatever libc produced. If you ever see a different hex, your headers or architecture packed _IOC differently. The kernel switch compares the full 32-bit word; a size mismatch is a different command.

3. Opening the device

open_dev() is open("/dev/vuln_lkm", O_RDWR). The path is "/dev/" VULN_LKM_NAME so it cannot drift from the miscdevice name. O_RDWR is conventional for ioctl; this driver does not implement .read or .write, so the mode mainly has to pass the inode permission check. load.sh chmods the node 0666 after insmod. If the module is not loaded, open fails with ENOENT. If permissions are 0600 and you are not root, open fails with EACCES. The CLI prints open /dev/vuln_lkm: <strerror> and exits 1. It does not retry, does not look for another node, does not fall back to /sys.

The file descriptor is held for the lifetime of main. Interactive mode reuses one fd for every menu pick. One-shot mode opens, issues one ioctl family, closes. There is no flock. Two CLI processes can race on g_val in the kernel; the lab is meant to be used one at a time.

4. Integer commands: GET, SET, ADD, SUB

All four share struct vuln_lkm_int { uint32_t value; } and the helper do_int(). do_int prints the struct (show_int), calls ioctl(fd, cmd, arg), and reports ioctl ok or ioctl failed: errno. The pointer passed to ioctl is the address of a local struct. The kernel will copy_from_user or copy_to_user that address. The CLI never inspects the kernel's g_val except by GET, which copy_to_user's into the same struct.

4.1 parse_u32 and strtoul base 0

Numbers are parsed with strtoul(s, &end, 0). Base 0 means a 0x prefix is hex, a leading 0 is octal, otherwise decimal. That is why the help text says <n> is decimal or 0xhex and why set 0xffffffffe works. Overflow of unsigned long is reported via errno. A leftover character after the number is a parse error. The result is truncated to uint32_t. Values larger than 2^32-1 on a 64-bit host will silently truncate; keep inputs in the u32 range.

4.2 GET

cmd_get zeros a vuln_lkm_int, calls do_int with sending=0 so the banner says number buffer rather than number sent, then if ioctl succeeded prints kernel value and a second base64 of the now-filled struct. Before ioctl the four bytes are 0. After ioctl they are whatever the kernel wrote. Compare the two b64 lines if you want to see the copy_to_user.

4.3 SET, ADD, SUB

Each constructs arg.value = v and calls do_int with sending=1. SET plants g_val. ADD asks the kernel to add. SUB asks the kernel to subtract. The CLI does not clamp, does not warn about wrap, does not read back unless you GET afterwards. Wrap is a kernel-side printk and a later GET. Example: set 10 then sub 11 then get should show 4294967295. set 0xffffffffe then add 3 then get should show 1.

5. STR command

cmd_str is the only path that uses struct vuln_lkm_str. It memset's the 260-byte object to zero, caps strlen(s) at VULN_LKM_STR_MAX (256), stores that as len, memcpy's the bytes into data[], prints show_str, then ioctl(VULN_LKM_STR, &arg). The CLI cap is 256, matching the ioctl blob, not 32. Sending a 40-character string is allowed by the CLI. The kernel memcpy's 40 bytes into a 32-byte kbuf. That is the lab.

show_str prints the C string with %. If data is not NUL-terminated inside the 256-byte array, %s would run off; memset of the whole struct plus a cap at 256 with no forced terminator at data[len] is almost safe because the remainder is zeros unless len == 256. A 256-byte payload with no room for a terminator can make %s read past data[]. The ioctl still sends 260 bytes of struct. Interactive mode uses fgets into a 256-byte buffer and strips a newline, so interactive strings are shorter than 256.

The 40-A example in --help is the intended overflow trigger: forty 0x41 bytes, len=40, kernel kbuf[32]. Run it in QEMU.

6. Base64 dump

print_b64 is a local encoder. Alphabet is the standard RFC 4648 table. It walks the buffer in 3-byte groups, emits 4 characters, pads with =. For GET/SET/ADD/SUB the payload is 4 bytes, so the b64 is 8 characters with padding. For STR the payload is the entire 260-byte struct (len plus 256 data bytes), so the dump is long. That is deliberate: you can paste the b64 into a decoder and see the little-endian len word followed by the string bytes.

Nothing about base64 is sent to the kernel. It is a display of the bytes the ioctl pointer refers to. If you ever need to prove what userspace handed copy_from_user, this dump is that proof.

7. Interactive loop

interactive() prints a seven-line menu and reads one line with fgets into 32 bytes. Only line[0] is the command. 1 GET, 2 SET, 3 ADD, 4 SUB, 5 STR, h help, q quit. SET/ADD/SUB call read_u32 which fgets a 64-byte line and parse_u32. STR fgets into a 256-byte buffer (VULN_LKM_STR_MAX), strips a trailing newline, and cmd_str. Unknown keys print unknown option. EOF (Ctrl-D) leaves the loop.

The menu text says overflow wrap and copy into 32-byte kernel buffer so the operator is not surprised. Help from h reprints print_help. The interactive path and the argv path share cmd_get / cmd_set / cmd_add / cmd_sub / cmd_str. There is no second implementation.

8. argv dispatch in main

Order of checks: help without open; open_dev or exit 1; argc==1 interactive; else exact matches: get with argc==2, set/add/sub with argc==3 and a successful parse, str with argc==3 (the string is argv[2], so a string with spaces must be quoted by the shell). Anything else prints help and returns 1. close(fd) always runs. A negative rc from ioctl helpers becomes exit status 1.

str with no argument, set with a non-number, extra arguments, and unknown verbs all fall through to help. There is no subcommand for dump-only or dry-run. There is no way to send a crafted len that differs from strlen; the CLI always sets len from the C string length. To send a mismatched len you would write a different program. This CLI is a tester, not a packet forger.

9. What the CLI does not do

- No privilege-escalation payload, no SMEP/KASLR bypass, no ROP.
- No raw ioctl number on the command line; only the five named ops.
- No read(2)/write(2) on the device (the driver has no .read/.write).
- No mmap, poll, or netlink.
- No retry on EINTR; a signal during ioctl surfaces as failure.
- No JSON, no scriptable machine output beyond the printed hex and b64.

10. Example sessions (userspace side)

These are the prints the CLI is written to produce. Kernel dmesg is a separate stream (see vuln_lkm.pdf and the run-debug manual).

10.1 GET after a fresh insmod

```

1 $ ./vuln_lkm_cli get
2
3 --- GET ---
4 ioctl cmd: 0x80041b01
5 number buffer: 0 (0x00000000)
6 payload b64: AAAAAA==
7 ioctl ok
8 kernel value: 100 (0x00000064)
9 returned b64: ZAAAAA==

```

g_val starts at 100. Little-endian 100 is 64 00 00 00, which is b64 ZAAAAA==. The pre-ioctl buffer was zeros (AAAAAA==).

10.2 Underflow wrap

```

1 $ ./vuln_lkm_cli set 10
2 --- SET ---
3 ioctl cmd: 0x40041b02
4 number sent: 10 (0x0000000a)
5 payload b64: CgAAAA==
6 ioctl ok
7
8 $ ./vuln_lkm_cli sub 11
9 --- SUB ---
10 ioctl cmd: 0x40041b04
11 number sent: 11 (0x0000000b)
12 payload b64: CwAAAA==
13 ioctl ok
14
15 $ ./vuln_lkm_cli get
16 kernel value: 4294967295 (0xffffffff)

```

10.3 Overflow wrap

```

1 $ ./vuln_lkm_cli set 0xfffffffffe
2 --- SET ---
3 ioctl cmd: 0x40041b02
4 number sent: 4294967294 (0xfffffffffe)
5 payload b64: /v///w==
6 ioctl ok
7
8 $ ./vuln_lkm_cli add 3
9 --- ADD ---
10 ioctl cmd: 0x40041b03
11 number sent: 3 (0x00000003)
12 payload b64: AwAAAA==
13 ioctl ok
14
15 $ ./vuln_lkm_cli get
16 kernel value: 1 (0x00000001)

```

10.4 STR that fits, then STR that does not

```

1 $ ./vuln_lkm_cli str hello
2 --- STR ---
3 ioctl cmd: 0x41041b05
4 string sent: "hello"
5 len sent: 5
6 payload b64: BQAAAGh1bGxv... # 260-byte struct, truncated here
7 ioctl ok
8
9 $ ./vuln_lkm_cli str AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
10 --- STR ---
11 ioctl cmd: 0x41041b05
12 string sent: "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
13 len sent: 40
14 ioctl ok
15 # kernel memcpy(40) into kbuf[32]; guest may oops. Use QEMU.

```

11. Error paths the CLI surfaces

open failure: printed to stderr, exit 1. ioctl failure: strerror(errno) on stdout, helper returns -1, main maps that to exit 1. Typical errno values: ENOTTY if the command word does not match the loaded module (header skew), EFAULT if the pointer were bad (the CLI's locals are always valid; you would need a different caller), ENODEV/ENXIO if the misc device vanished mid-run after rmmmod. A guest oops during STR may kill the CLI with a signal or hang the VM; that is why STR length tests belong in QEMU.

12. Complete source listing: vuln_lkm_cli.c

The file as committed. Helpers: print_help, print_b64, show_int, show_str, do_int, parse_u32, read_u32, open_dev, cmd_*, interactive, main.

```

1 #include <errno.h>
2 #include <fcntl.h>
3 #include <stdint.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7 #include <unistd.h>
8
9 #include "vuln_lkm.h"
10
11 #define DEV_PATH "/dev/" VULN_LKM_NAME
12
13 static const char b64tab[] =
14     "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";
15
16 static void print_help(const char *argv0)
17 {
18     printf(
19     "vuln_lkm_cli - userspace tester for the vuln_lkm educational LKM\n"
20     "\n"
21     "Usage:\n"
22     " %s interactive menu\n"
23     " %s -h|--help this help\n"
24     " %s get VULN_LKM_GET\n"
25     " %s set <n> VULN_LKM_SET\n"
26     " %s add <n> VULN_LKM_ADD (u32 wrap)\n"
27     " %s sub <n> VULN_LKM_SUB (u32 wrap)\n"

```

```

28 " %s str <string> VULN_LKM_STR (copy into 32-byte kbuf)\n"
29 "\n"
30 "<n> is decimal or 0xhex.\n"
31 "Every ioctl prints cmd hex, the number or string, and base64 of the payload.\n"
32 "Device: %s\n"
33 "\n"
34 "Examples:\n"
35 " %s get\n"
36 " %s set 10\n"
37 " %s sub 11 # 10-11 wraps to 4294967295\n"
38 " %s set 0xfffffffffe\n"
39 " %s add 3 # wraps to 1\n"
40 " %s str hello # fits in 32-byte kbuf\n"
41 " %s str AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA # 40 bytes > 32\n"
42 "\n"
43 "Interactive keys: 1 GET 2 SET 3 ADD 4 SUB 5 STR q quit\n",
44     argv0, argv0, argv0, argv0, argv0, argv0, argv0,
45     DEV_PATH,
46     argv0, argv0, argv0, argv0, argv0, argv0, argv0);
47 }
48
49 static void print_b64(const void *data, size_t n)
50 {
51     const unsigned char *p = data;
52     size_t i;
53
54     for (i = 0; i < n; i += 3) {
55         unsigned a = p[i];
56         unsigned b = (i + 1 < n) ? p[i + 1] : 0;
57         unsigned c = (i + 2 < n) ? p[i + 2] : 0;
58         unsigned triple = (a << 16) | (b << 8) | c;
59
60         putchar(b64tab[(triple >> 18) & 63]);
61         putchar(b64tab[(triple >> 12) & 63]);
62         putchar((i + 1 < n) ? b64tab[(triple >> 6) & 63] : '=');
63         putchar((i + 2 < n) ? b64tab[triple & 63] : '=');
64     }
65 }
66
67 static void show_int(const char *op, unsigned int cmd,
68                     const struct vuln_lkm_int *arg, int sending)
69 {
70     printf("\n--- %s ---\n", op);
71     printf("ioctl cmd: 0x%x\n", cmd);
72     printf("number %s: %u (0x%08x)\n",
73           sending ? "sent" : "buffer", arg->value, arg->value);
74     printf("payload b64: ");
75     print_b64(arg, sizeof(*arg));
76     printf("\n");
77 }
78
79 static void show_str(unsigned int cmd, const struct vuln_lkm_str *arg)
80 {
81     printf("\n--- STR ---\n");
82     printf("ioctl cmd: 0x%x\n", cmd);
83     printf("string sent: \"%s\"\n", arg->data);
84     printf("len sent: %u\n", arg->len);
85     printf("payload b64: ");
86     print_b64(arg, sizeof(*arg));
87     printf("\n");

```

```
88 }
89
90 static int do_int(int fd, const char *op, unsigned int cmd,
91                 struct vuln_lkm_int *arg, int sending)
92 {
93     show_int(op, cmd, arg, sending);
94     if (ioctl(fd, cmd, arg) < 0) {
95         printf("ioctl failed: %s\n", strerror(errno));
96         return -1;
97     }
98     printf("ioctl ok\n");
99     return 0;
100 }
101
102 static int parse_u32(const char *s, uint32_t *out)
103 {
104     unsigned long v;
105     char *end;
106
107     errno = 0;
108     v = strtoul(s, &end, 0);
109     if (end == s || *end != '\0' || errno)
110         return -1;
111     *out = (uint32_t)v;
112     return 0;
113 }
114
115 static int read_u32(const char *prompt, uint32_t *out)
116 {
117     char buf[64];
118
119     printf("%s", prompt);
120     fflush(stdout);
121     if (!fgets(buf, sizeof(buf), stdin))
122         return -1;
123     buf[strcspn(buf, "\n")] = '\0';
124     return parse_u32(buf, out);
125 }
126
127 static int open_dev(void)
128 {
129     int fd = open(DEV_PATH, O_RDWR);
130
131     if (fd < 0)
132         fprintf(stderr, "open %s: %s\n", DEV_PATH, strerror(errno));
133     return fd;
134 }
135
136 static int cmd_get(int fd)
137 {
138     struct vuln_lkm_int arg = {0};
139
140     if (do_int(fd, "GET", VULN_LKM_GET, &arg, 0) == 0) {
141         printf("kernel value: %u (0x%08x)\n", arg.value, arg.value);
142         printf("returned b64: ");
143         print_b64(&arg, sizeof(arg));
144         printf("\n");
145     }
146     return 0;
147 }
```

```
148
149 static int cmd_set(int fd, uint32_t v)
150 {
151     struct vuln_lkm_int arg = { .value = v };
152
153     return do_int(fd, "SET", VULN_LKM_SET, &arg, 1);
154 }
155
156 static int cmd_add(int fd, uint32_t v)
157 {
158     struct vuln_lkm_int arg = { .value = v };
159
160     return do_int(fd, "ADD", VULN_LKM_ADD, &arg, 1);
161 }
162
163 static int cmd_sub(int fd, uint32_t v)
164 {
165     struct vuln_lkm_int arg = { .value = v };
166
167     return do_int(fd, "SUB", VULN_LKM_SUB, &arg, 1);
168 }
169
170 static int cmd_str(int fd, const char *s)
171 {
172     struct vuln_lkm_str arg;
173     size_t n = strlen(s);
174
175     memset(&arg, 0, sizeof(arg));
176     if (n > VULN_LKM_STR_MAX)
177         n = VULN_LKM_STR_MAX;
178     arg.len = (uint32_t)n;
179     memcpy(arg.data, s, n);
180     show_str(VULN_LKM_STR, &arg);
181     if (ioctl(fd, VULN_LKM_STR, &arg) < 0) {
182         printf("ioctl failed: %s\n", strerror(errno));
183         return -1;
184     }
185     printf("ioctl ok\n");
186     return 0;
187 }
188
189 static int interactive(int fd)
190 {
191     char line[32];
192     uint32_t v;
193     char buf[VULN_LKM_STR_MAX];
194
195     while (1) {
196         puts("\nvuln_lkm CLI");
197         puts(" 1) GET integer");
198         puts(" 2) SET integer");
199         puts(" 3) ADD (u32 overflow wrap)");
200         puts(" 4) SUB (u32 underflow wrap)");
201         puts(" 5) STR (copy into 32-byte kernel buffer)");
202         puts(" h) help");
203         puts(" q) quit");
204         printf("> ");
205         fflush(stdout);
206         if (!fgets(line, sizeof(line), stdin))
207             break;
```

```
208     switch (line[0]) {
209     case '1':
210         cmd_get(fd);
211         break;
212     case '2':
213         if (read_u32("value to SET (dec or 0xhex): ", &v))
214             puts("bad number");
215         else
216             cmd_set(fd, v);
217         break;
218     case '3':
219         if (read_u32("value to ADD (dec or 0xhex): ", &v))
220             puts("bad number");
221         else
222             cmd_add(fd, v);
223         break;
224     case '4':
225         if (read_u32("value to SUB (dec or 0xhex): ", &v))
226             puts("bad number");
227         else
228             cmd_sub(fd, v);
229         break;
230     case '5': {
231         size_t n;
232
233         printf("string to send: ");
234         fflush(stdout);
235         if (!fgets(buf, sizeof(buf), stdin))
236             break;
237         n = strlen(buf);
238         if (n && buf[n - 1] == '\n')
239             buf[--n] = '\0';
240         cmd_str(fd, buf);
241         break;
242     }
243     case 'h':
244     case 'H':
245         print_help("vuln_lkm_cli");
246         break;
247     case 'q':
248     case 'Q':
249         return 0;
250     default:
251         puts("unknown option (h for help)");
252         break;
253     }
254 }
255 return 0;
256 }
257
258 int main(int argc, char **argv)
259 {
260     int fd, rc = 0;
261     uint32_t v;
262
263     if (argc >= 2 && (!strcmp(argv[1], "-h") || !strcmp(argv[1], "--help"))) {
264         print_help(argv[0]);
265         return 0;
266     }
267 }
```

```

268     fd = open_dev();
269     if (fd < 0)
270         return 1;
271
272     if (argc == 1) {
273         rc = interactive(fd);
274     } else if (!strcmp(argv[1], "get") && argc == 2) {
275         rc = cmd_get(fd);
276     } else if (!strcmp(argv[1], "set") && argc == 3 && !parse_u32(argv[2], &v)) {
277         rc = cmd_set(fd, v);
278     } else if (!strcmp(argv[1], "add") && argc == 3 && !parse_u32(argv[2], &v)) {
279         rc = cmd_add(fd, v);
280     } else if (!strcmp(argv[1], "sub") && argc == 3 && !parse_u32(argv[2], &v)) {
281         rc = cmd_sub(fd, v);
282     } else if (!strcmp(argv[1], "str") && argc == 3) {
283         rc = cmd_str(fd, argv[2]);
284     } else {
285         print_help(argv[0]);
286         rc = 1;
287     }
288
289     close(fd);
290     return rc < 0 ? 1 : rc;
291 }

```

13. How this maps to kernel functions

- cmd_get → ioctl GET → vuln_lkm_ioctl GET branch → copy_to_user g_val
- cmd_set → ioctl SET → store g_val
- cmd_add → ioctl ADD → vuln_lkm_add (u32 wrap)
- cmd_sub → ioctl SUB → vuln_lkm_sub (u32 wrap)
- cmd_str → ioctl STR → vuln_lkm_str (memcpy into 32-byte kbuf)

GDB on the guest kernel will stop in those four symbols if you follow [vuln_lkm_run_debug.pdf](#). The CLI itself you can also gdb on the host, but the interesting state (g_val, kbuf) is on the kernel stack and in module .bss.